

Speaking script — WireGuard for Apache NuttX

(Community Over Code, Glasgow)

Deck: [coc-glasgow-slides.html](#), 26 slides, English, current as of 2026-09-28 (post kernel-migration). This file is a printable copy of the notes already embedded in the deck (press `N` there to see them live) — use whichever is easier to rehearse from. If the deck is edited, re-export this file from those notes rather than editing the two independently; they will drift otherwise (see [../upstream/handoff.md](#) ground rule 5 on tests that cannot fail — the same logic applies to a script that cannot go stale).

Total: about 26 min, including a 90-second demo for a 26-slide, 20–30 min slot. Times below are cumulative, so read them as "should be at roughly this point."

The centre of gravity is slides 18–20 (verification): together they are 10.7 of the 26 minutes, and they carry the claims a reviewer will actually check. Do not compress that block to protect time elsewhere — cut before it or after it, never inside it. Slides 18-19-20 are also a locked sequence in meaning: 18 is where it runs, 19 is what "it runs" does not prove, 20 is the one place the answer is a judgment call rather than a test. Splitting them loses the argument.

If time is short, drop in this order: slide 23 (portability table — supporting evidence, not the spine), then slide 22 (operability). If the live demo stalls, switch immediately to the local recording. Keep slides 3–4: they explain what NuttX is and why this work exists. Do not drop anything in 8–19 or 24–25; those are the shape of the talk.

1 — Title (0:00 → 0:47)

On slide: "WireGuard for Apache NuttX." ESP32-S3 & SPRESENSE tunnel over real Wi-Fi · real WireGuard peers (Linux kernel, Windows client) · runs in a real kernel build (BUILD_KERNEL) · one source across NuttX 13.0.1 / master / 12.7.0.

Say:

- Today I will talk about bringing WireGuard to Apache NuttX.
- It is a WireGuard VPN implemented as a `wg0` network device. The tunnel is up over real Wi-Fi on ESP32-S3 and on SPRESENSE, the peers are always real WireGuard implementations — the Linux kernel module and the official Windows client. The apps version builds from one source on three NuttX versions. Later in the talk the new driver runs inside real KERNEL and PROTECTED builds, which is the harder and more interesting half.
- But the point of this talk is not that it works. It is what "works" hid, and what pushing it into the kernel forced into the open.

2 — Summary (0:47 → 1:14)

On slide: the `wg0` netdev diagram — application/NSH, NuttX network stack, `eth0/wlan0` next to `wg0`, a UDP socket, the internet, the peer.

Say:

- Here is the whole idea in one picture.
- Apache NuttX is a POSIX-compliant RTOS with its own TCP/IP stack. It had no VPN. This project adds `wg0` as a virtual network interface: applications see an ordinary interface, and underneath, the encrypted transport is a UDP socket. The peer on the other end is a real WireGuard endpoint — Linux, Windows, or another gateway.
- That POSIX and BSD-socket shape is why NuttX is a good fit. I am not claiming FreeRTOS or Zephyr cannot run a VPN; I am showing how naturally it becomes an ordinary network device in NuttX.

3 — Where NuttX Sits (1:14 → 2:10)

Say:

- Before the implementation, where NuttX sits.
- FreeRTOS, Zephyr and NuttX are all capable embedded operating systems; this is not a scorecard. FreeRTOS centers on a deliberately small kernel plus libraries. Zephyr is an integrated connected-device platform, with POSIX as an opt-in subset. NuttX starts from a Unix-shaped surface: POSIX and ANSI APIs, BSD sockets, filesystems and a shell, scaled down to microcontrollers.
- NuttX is also an Apache Software Foundation top-level project. Its kernel, applications and documentation are developed in public by the contributor community. That matters twice here: the Unix shape makes a VPN fit naturally as a network device, and the Apache project makes upstream review — not a desk demo — the definition of done.

4 — Why This Topic (2:10 → 3:30)

Say:

- The reason for this topic came from the devices I work with.
- I work as an edge AI engineer at Sony Semiconductor Solutions, using NuttX from the application side. One setting is AITRIOS edge AI: devices built around ESP32 and NuttX, deployed where walking up with a debug cable is not the operating model. The other is bottom-up work around SPRESENSE, Sony's compact low-power board used in remote sensing and space projects.
- I am not saying this WireGuard driver ships in either product. Those settings exposed the need: after a device is installed, how do I reach it with normal tools without exposing it, inventing a protocol, or requiring a vendor cloud? Wi-Fi security protects one radio hop. I wanted an authenticated path all the way to the MCU, over whatever network lies underneath.

5 — Why It Matters (3:30 → 4:01)

Say:

- It is not that you can do new things. It is that the things you already wanted to do become ordinary.
- Maintenance with the usual tools. Pulling the cloud into the device's network rather than the other way around. Zero trust that does not stop at a gateway but reaches the MCU. And because it is standard WireGuard, the other end can be anything — that interoperability is itself the value.

6 — The Plan (4:01 → 4:32)

Say:

- Writing a crypto stack from scratch would be a mistake. So I started from an existing implementation.
- `wireguard-lwip` implements WireGuard as an lwIP netif, and crucially it isolates everything OS-specific behind one header with four functions. The protocol and the crypto are portable C. NuttX does not use lwIP, though — it has its own stack. So the whole job is: keep the protocol core untouched, and replace the network glue.

7 — Design Decision (4:32 → 4:53)

On slide: lwIP → NuttX mapping table (`netif` → `net_driver_s`, `pbuf` → `iob`, `netif_add()` → `netdev_register()`, `callback` → `devif_poll()`).

Say:

- The single most important porting decision.
- Rather than drag lwIP into NuttX, I mapped the abstractions onto NuttX's own. `netif` becomes `net_driver_s`, `pbuf` becomes `iob`, `netif_add` becomes `netdev_register`, and the send callback becomes NuttX's `devif_poll`. `wg0` is registered as a TUN-type netdev, and the "wire" underneath it is a UDP socket.

8 — How It Was Built (4:53 → 5:23)

Say:

- The result of the first version.
- The protocol and crypto came across byte-identical — over three thousand lines unchanged. The four OS hooks are small. The real work was the network glue. And it worked: you can generate a key, set peers, load a config, all at run time, no rebuild. So far, a success story. The rest of the talk is about what that success hid.

9 — Part 2: The Pattern (5:23 → 5:45)

Say:

- This is the heart of the talk.
- Every difficult bug in this project had the same shape. A shallow test passes; a deeper one fails. Worse, the passing shallow test convinces you that area is fine, so you search everywhere else for hours. The next slides are instances of that pattern — five in the apps version, and then two more when I pushed it into the kernel.

10 — Pitfall 1/7: `SO_RCVTIMEO` (5:45 → 6:11)

Say:

- The first taste: a success return with no effect.
- Setting a receive timeout returned zero, success. But `recvfrom` still blocked forever — the option was accepted and silently ignored. That produced a perfect deadlock: the timer never fired, so no initiation was sent, so the peer never replied, so `recvfrom` never returned to let the timer fire.

11 — Pitfall 2/7: detached pthread (6:11 → 6:51)

Say:

- The second: a lifecycle assumption that did not hold.
- The receive thread was created detached. When the launching command exited, the thread stopped — it did not even show up in `ps`. A short sleep after creation let its `printf` run, so it really was created. My mistake was reading `detach` as "it will outlive the command" — `detach` only controls when the thread's resources are reclaimed, not whether it survives its launcher exiting. So when the command exited, the thread went with it. That is why the kernel version owns the thread in the driver instead.

12 — Pitfall 3/7: ping works, TCP dies (6:51 → 7:51) (*the good one*)

Say:

- My favourite bug in the project.
- On the same tunnel, ping was perfect and even a TCP handshake completed — then the connection died. Instrumenting the send path showed `EBADF`, bad file descriptor. The sends that worked all happened inside the receive task itself — handshake replies, keepalives, the ping echo, the SYN-ACK. Sends from any other task failed.
- The cause: in NuttX, file descriptors are scoped to the task group, not to the whole system, and not global. The fix was to stop going through fds entirely and use NuttX's internal socket API, which operates on a `struct socket` — just memory — usable from any task. This is exactly why the kernel version holds the socket as a `struct socket`.

13 — Pitfall 4/7: "no real impact" was 10× (7:51 → 8:41)

Say:

- The fourth: an assumption I did not measure.
- The fd-less fix also broke `poll`, which is fd-based, so as a stopgap I polled the socket every fifty milliseconds. I assumed the only cost was waking the CPU when idle. Wrong. Every arriving packet waited up to fifty milliseconds, and that alone capped throughput — about ten times slower than it should be. I measured "no impact" and it was ten x. The real fix was a proper blocking wait: a callback on `psock_poll` and a semaphore.

14 — Pitfall 5/7: sim passes, hardware freezes (8:41 → 9:27)

Say:

- The fifth, and it comes back later: stacks nobody measured.
- Persisting the config by redirecting into a file froze the board, but only on real hardware's flash filesystem, never in sim. The cause was the receive task's stack. It calls into the IP stack on its own stack, so the whole TCP call depth lands there, and for packets it answers immediately, the reply encryption too. The default 3072 bytes was a number nobody had measured. Stack coloration plus `ps` found it. Hold that thought.

15 — Part 3: Into the kernel (9:27 → 10:02)

Say:

- This is the additional work since the first version, and the part that makes it mergeable.
- Version one lives in `apps` and reaches into socket internals. That is fine for a flat build, but it can never be a proper in-tree driver, and it does not fit the protected and kernel builds where user space and kernel are separated. So I moved the device into the kernel, under `drivers/net/wireguard`, and drove it from a tiny user-space `wg` command through `ioctl`.

16 — The Kernel Design (10:02 → 10:34)

Say:

- The shape of the real driver.
- `wg0` is now a lower-half netdev in the kernel, owning the UDP socket and a receive kernel thread — so the socket-ownership and launcher-lifetime problems from part two go away by design. Configuration is a flat, pointer-free `ioctl` ABI, pointer-free because the kernel copies the caller's structure directly. The private key is write-only: the get `ioctl` never hands it back — though it still lives in the config file on the user side. The kernel-side crypto is NuttX's own; the only vendored code is a small MIT X25519 in the user-space command, for offline key generation.

17 — Two more bugs, same shape (10:34 → 11:41) (*locked with 16*)

Say:

- And the same pattern held at kernel depth — twice.

- First, a bug in NuttX's own ChaCha20-Poly1305: the counter went into the wrong bytes of the nonce. For counter zero the two layouts are identical, so the handshake completes and then every data packet after the first fails to decrypt. Shallow passes, deep fails. There was no in-tree caller, so it had never been exercised. A two-line fix, and it is going upstream as its own `crypto`: PR — porting into the project found a latent bug in the project.
- Second, a crash that only happened in a kernel build. Setting the key snapshotted the peers into a six-kilobyte array on the stack, and the kernel stack is three kilobytes — overflow, heap corruption, panic. On sim, with large task stacks, it never showed. Exactly the stack lesson from part two, one level deeper. Moved to the heap.

18 — Verification: seven places it has to hold (11:41 → 15:52) (*core — do not cut*)

On slide: table of sim / rv-virt knetnsh64 (BUILD_KERNEL) / rv-virt pnsh64 (BUILD_PROTECTED) / rv-virt knetnsh64_smp (4 CPUs) / SPRESENSE measured / ESP32-S3 + SPRESENSE over real Wi-Fi / the two boards on apps v0.1.1.

Say:

- Be precise about which implementation was verified where.
- The kernel driver builds clean in the sim config — the configuration CI compiles — and it runs as a full tunnel in a real kernel build: rv-virt's `knetnsh64` in QEMU, over virtio-net to Linux kernel WireGuard, with the `wg` command loaded as a separate ELF and the driver reached across the syscall boundary.
- And then the other privilege model. The `ioctl` ABI is a fixed struct with no pointers inside it, so that the kernel never has to reach into user memory — and the way an assumption like that fails is not a compile error, it is an MPU exception on the first call. PROTECTED had only ever been built, on a board with no emulator. So I brought it up on rv-virt's protected config: the split boots, the `wg` command lives in the user blob, every configuration `ioctl` crosses the boundary, and the tunnel reaches Linux kernel WireGuard with no exception, no assertion, no panic. The memory map is the stock one — the kernel takes eighty percent of its side, the user command eight percent of the other.
- And the locking. The driver puts its protocol state under the device lock, not under there being only one CPU — and until this week that was an argument I made from the code rather than something I had contended. So: four CPUs, a two-thousand-packet flood arriving while another CPU is running configuration `ioctls` and a down-up cycle. No exception, no assertion, and the tunnel came back. One scripted interleaving on an emulator is not a race search, so a race that needs a specific window can still be hiding. But the claim is no longer only on paper.
- Last row, and the one I would put in front of a reviewer first. The sim soak said the resources come back exactly to where they started. Fine, but the simulator's allocator is the host's and its frames are sixty-four bit, so none of the heap numbers transfer to a board. So I measured the board: a hundred and twenty seconds of load, then forty down-up cycles, each one closing and reopening a socket that belongs to the Wi-Fi module through `usrsock` and re-handshaking over real Wi-Fi. Heap, allocation count, buffer pool, every stack high-water: identical before and after. And the number I actually wanted, the receive thread's deepest observed use, is one thousand four hundred and

seventy-two bytes against a six-kilobyte default. That default used to be a guess. It is a measurement now.

- And it now runs on real hardware — both boards. The kernel driver flashed to an ESP32-S3, tunnelling to the official Windows WireGuard client over real Wi-Fi; and to a SPRESENSE, whose GS2200M Wi-Fi is a usrsoc device, so the driver's sends really do go out through usrsoc on hardware. And then the fault case that design exists for was forced on that board: a datagram held outstanding on the real usrsoc backend while control operations arrive. A query answered, a peer update was accepted, the stop reported a timeout instead of hanging, the held ciphertext was untouched, and a repeated down reaped it so the interface came back. What that still does not cover is IOB exhaustion, losing Wi-Fi mid-send, and sustained load. The older apps version on the two boards — telnet, HTTP, a seven-megabyte transfer, rekey and power-cycle recovery — still stands underneath.

19 — Verification: a working tunnel is not evidence (15:52 → 18:22) (*core — do not cut*)

On slide: three rows — same private key every boot / session keys outliving the tunnel / a leak that only shows after hours — each with "does a passing ping notice?" answered "no."

Say:

- The previous slide was where it runs. This one is what a working tunnel cannot tell you.
- Take key generation. NuttX can provide `/dev/urandom` through an xorshift128 backend whose seed material is compile-time configuration. A board left on that path hands out the same private key on every single boot. The handshake works. The ping works. The demo works. The tunnel is worthless, and nothing I showed you so far would notice. So: reset the board three times, read one key each time, compare — three different keys, and the config is on the entropy pool rather than xorshift128. That is a useful regression check, not a certification of entropy quality.
- Second, what survives bringing the interface down. Here I stopped reading the code and read the memory: find the device in the running process by matching the key I set, then check the secret fields by name, at offsets computed from the driver's own headers. The transport keys are fully populated while the tunnel is up and every byte is zero after down, and the handshake state is already gone while it is up, because it is wiped the moment the keys are derived. The static key is still there — on purpose, because up has to work again — and there is exactly one copy of it in the whole writable address space, before and after. That asymmetry is a decision, so it is now written in the driver's documentation rather than left for a reviewer to guess at.
- Third, the soak. A hundred and fifty-one rekeys forced past the point where sessions expire, ten endpoint moves, a hundred down-up cycles, and the resources sampled throughout: the buffer pool comes back whole, the live allocation count returns to exactly what it was at boot — not close, equal — and the receive thread's stack high-water mark is two and a half kilobytes and does not move after the first sample. That last number is worth having on its own: the default stack for that thread is six kilobytes, so the margin is real rather than assumed. What I am not claiming is a week of uptime, or that a simulator's allocator and 64-bit frames behave like the board's.

20 — A design call: the timestamp problem (18:22 → 22:08) (*core — do not cut*)

On slide: the replay-defence obligation, the four-option table (realtime + in-boot high-water / persist every timestamp / durable range reservation / let it catch up), and the measured pair (clock unset → no handshake in 75 s; clock set → 4.1 s).

Say:

- This is the one where I had to decide rather than test. Then I went and measured the decision.
- WireGuard's replay defence puts an obligation on the initiator: every handshake must carry a timestamp greater than anything that peer already accepted, and that has to hold across a reboot. The device is usually the one dialling out, and it is the one least likely to have a battery-backed clock.
- The reproduction said recovery took seventy-five seconds, and that number is misleading. The device has to climb back past the last handshake of its previous session, so the outage is as long as the previous uptime. Seventy-five seconds only because that session was short. Something that ran for a month cannot reconnect for a month. That kills the do-nothing option outright.
- Writing the timestamp every handshake means a flash write every couple of minutes, forever, on the path that most needs to stay quick. Reserving a range durably and spending it from RAM is the right long-term answer, and I have written it up — but it needs a storage backend that will state its ordering and durability guarantees, and I could not get one. Locally, `hostfs sync` calls a void function and returns OK, so a successful write in the simulator proves nothing about power loss. Shipping that would look solved without being solved.
- So: realtime plus a high-water mark within the boot, which is exactly what Linux and `wireguard-go` do, and never fall back to uptime. What I added is the part a desktop never needed, because a desktop always has a clock. Without an RTC, NuttX seeds the clock from a configured year, so if the clock is still below the year after it, it was never set — and the driver says so once, naming the consequence. It still issues the timestamp, because a peer meeting this key for the first time will accept it and refusing would break first pairing on exactly those boards. The failure I am turning into one line of console output is not a crash. It is a tunnel that quietly never comes back, on a device that is not on your desk.
- And I did measure it, on the board, because a design argument you cannot demonstrate is just an opinion. The trap is that the obvious test passes. Reset the board, ping it, and the tunnel is back in sixteen seconds — because your ping made the *peer* initiate, and a responder needs no timestamp of its own. The device can be completely unable to dial out and that test still goes green.
- So: send the board nothing at all, and watch its own view of the session. Two runs, same board, same peer, same image, one variable. Clock as it boots: the driver warns, and there is no handshake after seventy-five seconds. Clock set from my laptop: the warning goes quiet, and the handshake completes in four point one seconds. That is both halves at once — the uncovered case is real on hardware, not just in a simulator, and the design is right wherever a clock exists. Fourteen stays open because I decided it should, not because I ran out of evidence.

21 — The Demo (22:08 → 23:38)

Say:

- The point of the demo is that there is nothing special to see.
- I telnet into the board through the tunnel and run commands, and I start a web server on the board and open it in a browser. The USB cable is unplugged; the board is on a power adapter. The peer is the official Windows client over home Wi-Fi. The board is just an ordinary host at the far end of an encrypted tunnel — which is exactly the goal.
- Let the commands breathe. Show `wg show` before and after loading the page so the byte counters visibly move; the change is the proof, not terminal noise.

If live: `uname -a / ifconfig (wg0 = 10.10.0.2) / wg show (handshake + byte counters) / ps (wg_rx running) / webserver &`, then browse to `http://10.10.0.2/`. **Do not show** `.config`, `kconfig-tweak` output, build logs, or `wg showconf` (prints the private key) on screen — the SSID and passphrase are plaintext in those places. **Fallback:** the recording, youtu.be/1kyX2av5WG4.

22 — Operability (23:38 → 23:59)

Say:

- Moving from "it works in a demo" to "you could run it."
- Keys are no longer baked into the build — you generate and set them at run time, and they never end up in the firmware image. And for long runs I sample four independent signals every minute — is the router side alive, is the board on the network, is WireGuard carrying, is the application data flowing — so when it goes quiet I can say which layer failed instead of guessing.

23 — Portability (23:59 → 24:23) (*cut candidate*)

Say:

- The design decision from the start, validated.
- Isolating the OS behind a few functions was a bet. Here is the check, with the apps FLAT implementation: four architectures — x86, ARM Cortex-A7, Xtensa on the ESP32, and Cortex-M4F on SPRESENSE — with no code change to move between them. The kernel driver then adds sim and the rv-virt kernel build on top. The bet paid off.

24 — Contributing Back (24:23 → 24:47)

Say:

- Closing the loop with this room's values.
- The contribution is staged: a small `crypto`: PR first, for the nonce fix, because that stands on its own and helps everyone. Then the driver PR for the kernel device, and then the command PR for the user-space tool. The three signed candidate branches are published on my forks, and the patch-form

checks pass. The design discussion and upstream PRs are next; they are not open yet. Community over code means letting that review change the work.

If asked "is it merged yet": no — this is a submission candidate, not a merge-ready declaration. See the Q&A doc.

25 — Takeaways (24:47 → 25:53)

Say:

- Three things to take away.
- First, shallow tests pass and deep tests fail — so build your tests for depth: sustained traffic, real hardware, a real kernel build, not just a handshake. Second, a clean OS-abstraction boundary is worth the effort — four architectures and three versions with no changes. Third, porting into a project is a chance to improve it. One bug was in NuttX's own crypto, and that fix goes upstream on its own. The second I only found by insisting my board image be reproducible from a clean checkout: a stock release tag does not boot to a shell on this board, because before the RTC comes up the clock reads zero and the watchdog that would finish bringing the RTC up never fires. And a third was in my own driver, where only a real kernel build surfaced it; sim never would have.

26 — Thank you (25:53 → 26:04)

Say:

- Thank you. I am happy to take questions — about the kernel driver, the bugs, the ioctl ABI, or the hardware.

Then: open [coc-glasgow-qa.md](#) alongside the deck for Q&A, not the slides themselves — several likely questions (rebase status, #14, IOB exhaustion, checkpatch) are answered there but are not on any slide.